

Predicting In-Hospital Mortality from the First 48 Hours of ICU Data

Michelle Carrizosa, M.S.

BSTT 550 project, Spring 2024
Revised October 2026

Abstract

Gradient-boosted trees were the strongest of eight classifiers for predicting in-hospital death from the first 48 hours of ICU data, but at the default 0.5 threshold they caught only about 1 in 10 deaths. I used the PhysioNet/Computing in Cardiology Challenge 2012 data: 11,988 ICU stays from the MIMIC-II database, 37 time-stamped clinical variables, and a 14.2% in-hospital mortality rate. Each variable was binned into hourly features (about 1,700 in total) and modeled for the full cohort and separately for four ICU types. On a held-out 20% test set, the best overall model (gradient boosting) reached a ROC-AUC of about 0.84. Its accuracy of 0.867 was barely above the 0.862 a model gets by predicting that every patient survives, and its recall for deaths was 0.11. SHAP values pointed to age, Glasgow Coma Score late in the 48-hour window, urine output, and weight as the strongest predictors. The main lesson is that accuracy hides the problem in imbalanced clinical prediction; discrimination, calibration, and recall at a clinically chosen threshold are the right yardsticks.

1 Introduction

The question is whether routinely collected ICU data from a patient’s first two days can flag who is at high risk of dying in the hospital. An early, reliable risk signal lets care teams prioritize review, escalate monitoring, or start goals-of-care conversations sooner. Acuity scores such as SAPS-I and SOFA already estimate severity with a fixed formula and are widely used to adjust for population differences when comparing treatments. Machine-learning models can instead use the full, irregular time series to make patient-specific predictions.

The PhysioNet/Computing in Cardiology Challenge 2012 framed exactly this task: predict in-hospital death using only the first 48 hours of each ICU stay [1]. The challenge scored entries on sensitivity, positive predictive value, and calibration, and emphasized generalizability across ICU types. I used the challenge data but made my own train/test split rather than the official one. I asked three questions:

1. Which classifier family discriminates best between survivors and non-survivors?
2. Does performance differ when models are fit separately for each ICU type?
3. Which variables, and which hours, drive the predictions?

2 Data

The cohort is 12,000 adult ICU stays of at least 48 hours, pooled from the challenge’s sets A, B, and C and drawn from MIMIC-II [2]. After merging the time series with outcomes, 11,988 stays had usable measurements. Each stay is one record.

Descriptors. Six values are recorded at admission: record ID, age, gender, height, weight, and ICU type. Missing values are coded as -1 . Gender was 44% male and 56% female. Table 1 shows the mix of ICU types.

Table 1: Distribution of ICU types (12,000 stays).

ICU type	Share of stays
1. Coronary care unit	14.7%
2. Cardiac surgery recovery unit	21.1%
3. Medical ICU	35.8%
4. Surgical ICU	28.4%

Time series. Thirty-seven variables are recorded with a timestamp relative to ICU admission. They include vitals (heart rate, blood pressure, temperature, respiration), labs (BUN, creatinine, lactate, troponin, and others), Glasgow Coma Score (GCS), urine output, and mechanical ventilation. A variable may be recorded many times, once, or never for a given stay, and timestamps differ across patients.

Outcome. The label is in-hospital death (0 = survived, 1 = died). The challenge defines it from survival days and length of stay: a death recorded within the hospital stay counts as in-hospital. Of the 12,000 stays, 14.2% ended in in-hospital death. SAPS-I, SOFA, length of stay, and survival days are also provided. I kept them out of the feature set, since length of stay and survival are only known after the outcome.

3 Data Exploration

Missingness is heavy and uneven (Figures 1 and 2). Vitals and core labs such as heart rate, GCS, BUN, and creatinine appear in about 98% of stays. Others are sparse: lactate in 55%, albumin in 41%, respiratory rate in 28%, cholesterol in 8%, and troponin-I in 5%. Invasive blood pressure (about 70%) and its non-invasive counterpart (about 87%) partly substitute for each other, since patients with an arterial line rarely need a cuff.

The data also contain physiologically impossible values, including a temperature of -17.8°C , a pH of 735, and blood pressures of 0. These look like unit errors or placeholders. They remained in the modeling data; Section 7 describes how I would handle them.

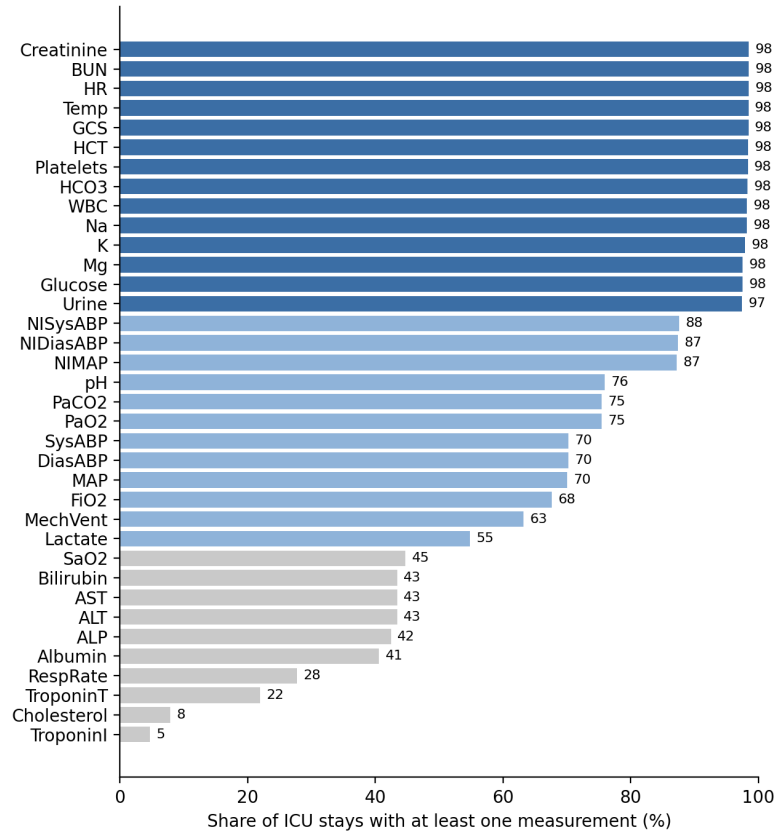


Figure 1: Share of ICU stays with at least one measurement of each time-series variable. Dark bars: at least 90% coverage; light bars: 50–90%; grey: under 50%.

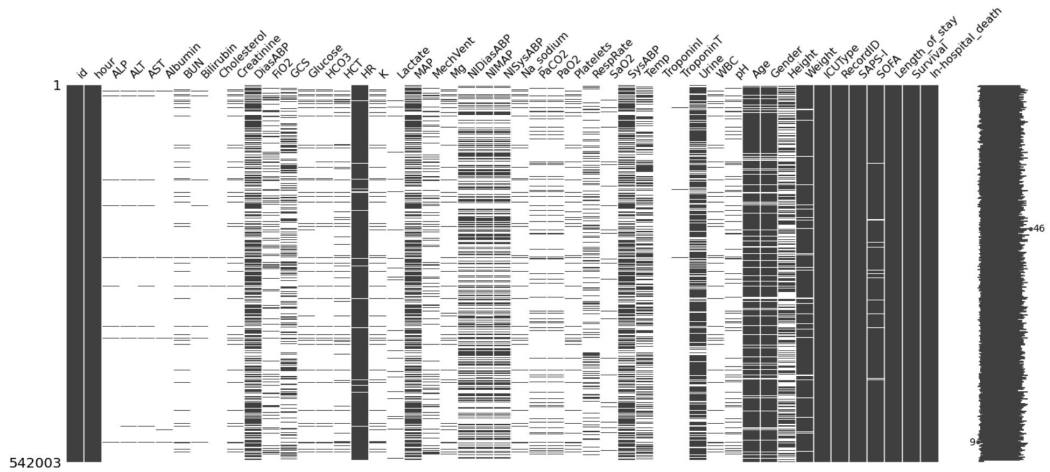


Figure 2: Missingness matrix of the long-format data (one row per patient-hour). Dark cells are observed values; white cells are missing.

4 Data Preparation

Feature construction. I bucketed each measurement by the hour since admission (hours 00–47). Then I pivoted to one row per stay and one column per variable-hour pair, such as `GCS_45`, giving about 1,760 candidate columns. I built two versions of these features:

- **Value features:** the mean of each variable’s measurements within each hour.
- **Frequency features:** the number of measurements of each variable within each hour. This captures how intensively a patient was monitored, which can itself signal severity.

I dropped columns missing in more than 80% of stays for the value features (90% for the frequency features), leaving 1,728 time-series columns. Age, gender, height, and weight were added as static features. For a patient with several recorded weights, I used their mean.

Missing data. Remaining gaps in time-series columns were filled with 0. Gaps in height and weight were filled with the cohort mean (for example, 169.5 cm for height).

Training and testing data. I fit models on the full cohort and on each ICU type separately. Each cohort was split 80/20 into training and test sets with a fixed random seed, rather than using the challenge’s original split. I used 5-fold cross-validation on the training set to check the stability of random forest accuracy.

5 Methods

I compared eight classifiers in Python using scikit-learn [3] and XGBoost [4]. I also fit penalized logistic models (lasso, ridge, elastic net, and sure independence screening with a lasso penalty) and CART. Hyperparameters and trade-offs are listed in Appendix A.

5.1 Logistic regression and penalized variants

Logistic regression models the log-odds of death as a linear function of the features. With about 1,700 correlated features, penalized versions are more stable: the lasso (L_1) shrinks many coefficients to zero, ridge (L_2) shrinks them toward zero, and the elastic net mixes the two. Sure independence screening first ranks features by marginal association and keeps the top set before fitting the lasso.

5.2 Tree-based models

A single decision tree splits patients into groups by threshold rules and is easy to explain but unstable. A random forest averages 100 trees, each grown on a bootstrap sample with a random subset of features. Gradient boosting (GBM) and XGBoost build shallow trees sequentially, each one correcting the errors of the trees before it; XGBoost adds regularization and native handling of missing values.

5.3 Other classifiers

I also fit a linear support vector machine, Gaussian naive Bayes, and a neural network (multilayer perceptron) as comparison points.

5.4 Evaluation

For each model I recorded accuracy, precision, recall, F1, ROC-AUC, and the confusion matrix on the test set. Because only 14% of patients died, I compare accuracy against a no-skill baseline that predicts survival for everyone. For XGBoost, I compared train and test accuracy across learning rates from 0.01 to 0.96 and kept 0.1 (Figure 9).

5.5 Interpretation

I used impurity-based feature importance from the tree ensembles and SHAP values [5] from the gradient-boosted model. SHAP assigns each feature a contribution to an individual prediction on the log-odds scale, using a game-theoretic approach.

6 Results

6.1 Discrimination

On the full cohort with value features, the gradient-boosted model’s ROC curve gave an AUC of 0.84 (Figure 6). The penalized logistic models clustered close behind (Figure 3). Frequency features alone discriminated less well: GBM reached 0.776 and a single decision tree 0.578. How often a patient was measured carries real signal, but the measured values add more.

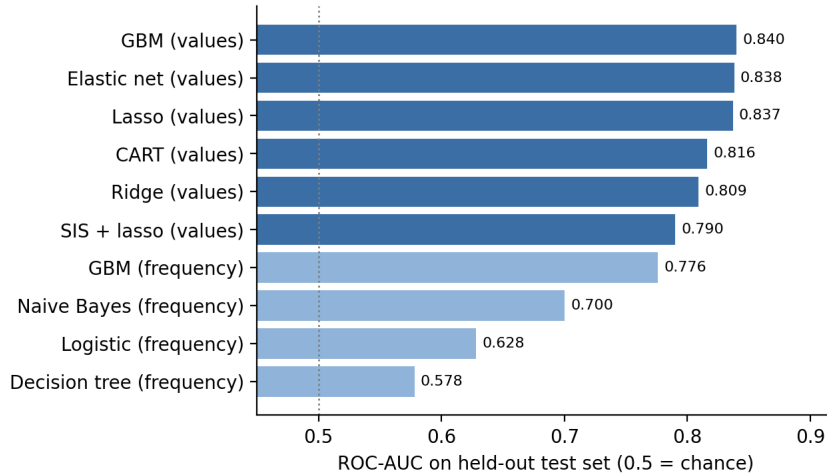


Figure 3: Test-set ROC-AUC by model and feature set. Dark bars use measured values; light bars use measurement counts only. The dotted line marks chance (0.5).

6.2 Accuracy by model and cohort

Table 2 reports test-set accuracy with value features. Predicting “survived” for every patient scores 0.862 on the full test set and 0.895 in ICU 1, so most models barely beat, or fall below, a rule that never predicts death (Figure 4). The high ICU 2 accuracies mainly reflect low mortality after cardiac surgery.

Table 2: Test-set accuracy by model and cohort (value features). The no-skill baseline is 0.862 overall and 0.895 in ICU 1.

Model	Overall	ICU 1	ICU 2	ICU 3	ICU 4
XGBoost	0.871	0.901	0.953	0.790	0.865
Gradient boosting	0.867	0.901	0.951	0.789	0.865
Random forest	0.861	0.870	0.952	0.813	0.853
Logistic regression	0.857	0.858	0.937	0.762	0.842
Neural network	0.846	0.904	0.933	0.776	0.773
Linear SVM	0.847	0.785	0.915	0.728	0.761
Naive Bayes	0.815	0.654	0.800	0.674	0.724
Decision tree	0.772	0.790	0.909	0.727	0.801

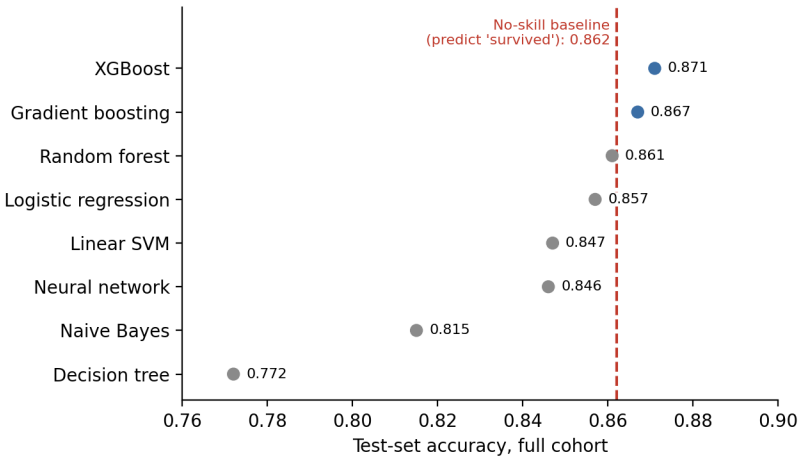


Figure 4: Full-cohort test accuracy against the no-skill baseline (dashed line).

6.3 Threshold metrics

At the default 0.5 probability cutoff, both boosted models found few deaths (Figure 5). Of 330 deaths in the overall test set, gradient boosting flagged 35 (recall 0.11, precision 0.59). XGBoost had the same recall with precision 0.70. Survivors were classified almost perfectly (recall 0.99). In ICU 1, the linear SVM had the best death recall (0.41) despite lower accuracy.

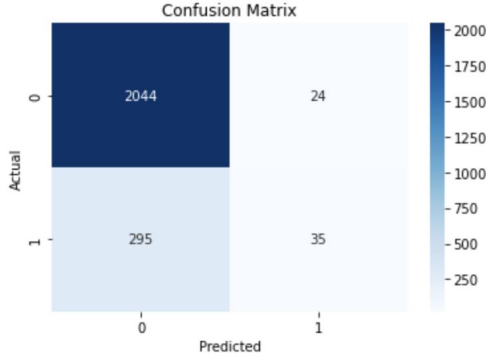


Figure 5: Confusion matrix, gradient boosting, full-cohort test set ($n = 2,398$).

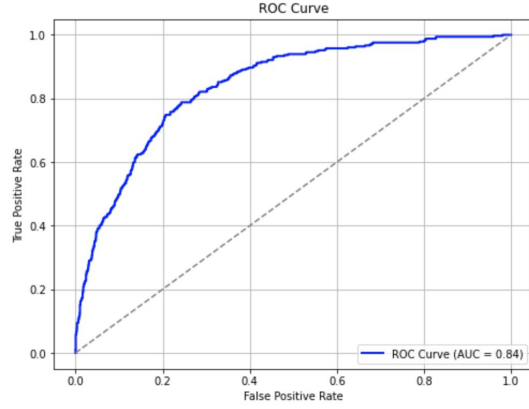


Figure 6: ROC curve, gradient boosting, full-cohort test set (AUC = 0.84).

6.4 Predictors

SHAP values (Figure 7) and impurity-based importance (Figure 8) agreed on the top drivers:

- **Age**, by a wide margin. Older age pushes risk up.
- **Glasgow Coma Score**, especially at hours 42–46. Higher (better) scores lower predicted risk.
- **Urine output** at hours 4, 38, and 47. Low output raises risk.
- **Weight**. Lower weight raises predicted risk.
- **Blood pressure at admission** (non-invasive systolic and mean, hours 0–1), **BUN**, **respiratory rate**, **lactate**, and **mechanical ventilation** in the second day.

The SHAP rankings varied by ICU type, which I did not explore in depth.

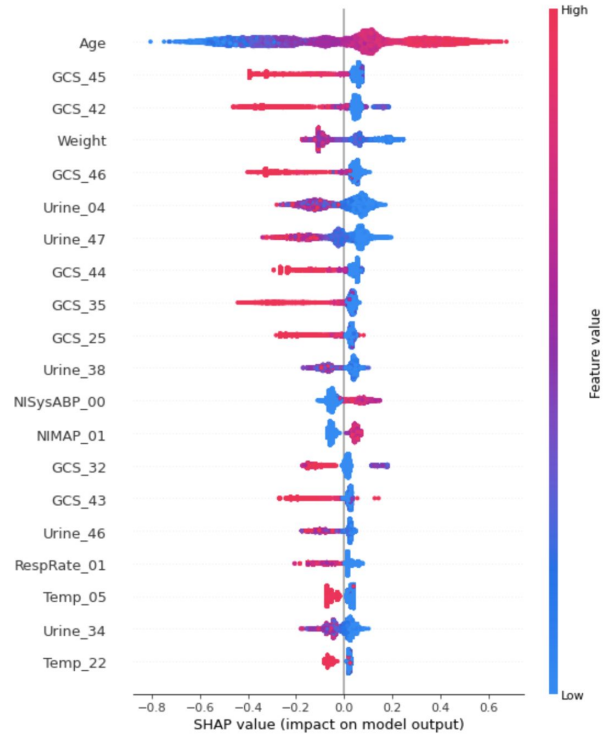


Figure 7: SHAP summary for the gradient-boosted model. Each dot is a patient; position shows the feature’s effect on predicted log-odds of death, and color shows the feature’s value (red high, blue low).

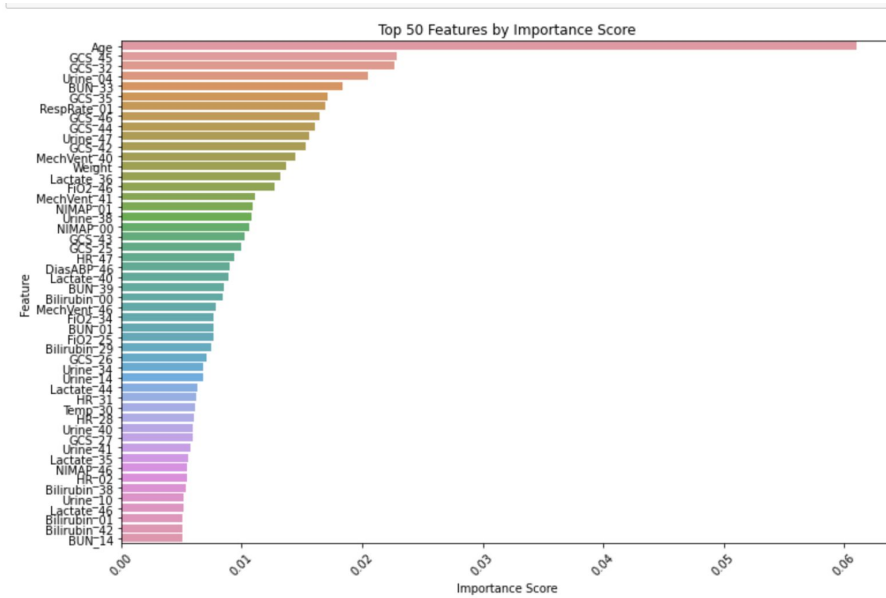


Figure 8: Top 50 features by impurity-based importance, gradient-boosted model.

7 Discussion

Boosted tree ensembles ranked patients by risk reasonably well (AUC about 0.84), but as built they would not work as an alert. With 14% mortality, a model can look 86% accurate while missing nearly every death. A clinical tool would set its threshold by the trade-off care teams accept, for example catching 70% of deaths at an acceptable false-alarm rate, rather than at 0.5.

What the predictors suggest. Age, consciousness (GCS), kidney output, and early blood pressure are clinically sensible drivers. The prominence of late-hour features (GCS at hours 42–46, urine at hour 47) likely reflects two things. Measurements closer to the 48-hour mark describe the patient’s state nearest the outcome. And because missing values were filled with 0, a late-hour feature also encodes whether the patient was still being assessed at that hour.

7.1 Limitations and what I would change

Table 3 lists the main weaknesses of this analysis and how I would address each. Separately, two results in the original notebooks should not be cited: a random forest showing accuracy and AUC of 1.0 was scored on data it had been trained on, and another random forest was fit on the test set itself.

Table 3: Limitations and proposed fixes.

Issue	Why it matters	Fix
Accuracy used as the headline metric	Imbalance makes accuracy near-meaningless; most models barely beat “everyone survives”	Lead with ROC-AUC and PR-AUC, plus recall and precision at a chosen threshold
Default 0.5 threshold, no class weighting	Recall for deaths was 0.11	Use class weights or <code>scale_pos_weight</code> ; choose the threshold on validation data
Missing values filled with 0	A creatinine or pH of 0 is impossible; mixes “not measured” with a real value	Per-variable summaries with missingness indicators, or last value carried forward
Implausible values and –1 codes left in	Temperatures of -17.8°C and pH of 7.35 add noise	Physiological range filters; recode –1 as missing
About 1,700 sparse hourly features	Most cells are empty; models overfit, especially in small ICU subsets	Summarize each variable as first, last, min, max, mean, count, and trend
Learning rate chosen by test accuracy	The test set informs model selection, so its score is optimistic	Tune with cross-validation on training data; score the test set once
Record ID included as a feature	IDs track data source and order, not patient risk	Drop it from the feature matrix
Unscaled inputs for logistic regression and SVM	Logistic regression hit its iteration limit	Standardize features inside a pipeline
One random split; small subsets (ICU 1 test set: 353 stays, 37 deaths)	Single-split AUCs have wide uncertainty	Bootstrap confidence intervals or repeated cross-validation
No severity-score benchmark	Can’t tell whether models beat existing tools	Report the AUC of SAPS-I alone on the same test set
No calibration check	Clinicians act on probabilities, not ranks	Calibration plots and the Brier score

8 Conclusion

First-48-hour ICU data supports moderate discrimination of in-hospital death (AUC about 0.84) with gradient-boosted trees, and the strongest signals are age, GCS, urine output, and early blood pressure. The models are not yet usable as alerts: at the default threshold they miss about 90% of deaths. The next version would use summary features with missingness indicators, class weighting with a validated threshold, a SAPS-I benchmark, and calibration and confidence intervals for every cohort. It would also extend SHAP to patient-level explanations and compare drivers across ICU types.

A Model settings and trade-offs

Table 4: Hyperparameters, strengths, and weaknesses of each classifier.

Model	Settings	Strength	Weakness
Gradient boosting	100 trees, learning rate 0.1, max depth 3	High accuracy from many weak learners; feature importance	Sensitive to outliers
XGBoost	100 trees, learning rate 0.1, max depth 3, $\gamma = 0.1$	Handles missing data natively; regularized	Longer training; weaker on small subsets
Random forest	100 trees	Handles high-dimensional data and outliers	Struggles with imbalance unless weighted
Logistic regression	scikit-learn defaults	Fast and interpretable	Needs scaled inputs; sensitive to outliers
Decision tree	scikit-learn defaults	Easy to explain	Less accurate than ensembles
Linear SVM	Linear kernel, $C = 1$	Works when features outnumber rows	Computationally expensive
Naive Bayes	Gaussian	Simple and fast	Assumes independent features
Neural network	1 hidden layer of 6 neurons, 100 iterations (earlier: 100 neurons, 1,000 iterations)	Flexible on large data	Needs large training sets; hard to interpret

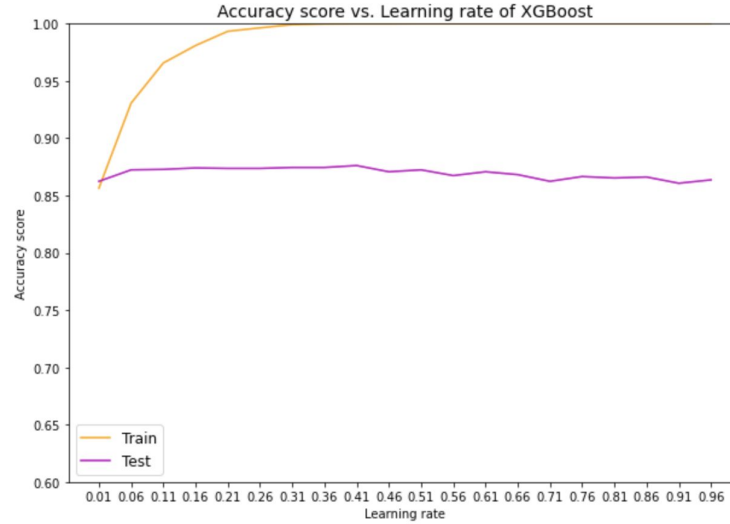


Figure 9: XGBoost train and test accuracy across learning rates. Training accuracy reaches 1.0 quickly while test accuracy stays flat, a sign of overfitting at higher rates.

References

- [1] I. Silva, G. Moody, D. J. Scott, L. A. Celi, and R. G. Mark. Predicting in-hospital mortality of ICU patients: The PhysioNet/Computing in Cardiology Challenge 2012. *Computing in Cardiology*, 39:245–248, 2012.
- [2] A. L. Goldberger et al. PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000.
- [3] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [4] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [5] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, 2017.